

UCS503P – Software Engineering Project

Meridian: Latent Cartographer

A Local-First Spatial Knowledge Engine and Vector Workspace

Submitted by:

1024160014 Rahul Dadwal

1024160133 Jyotsna Sachdeva

1024160130 Bisman Singh Rai

Group: 5

Bachelor of Engineering, Computer Science and Engineering

Submitted to:

Dr. Jeelani Asif

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

Executive Summary	3
1 Introduction	4
1.1 Background and Context	4
1.1.1 Problem Statement	4
1.1.2 Intended Users	4
1.1.3 Proposed User Journey	5
1.2 Project Objectives	5
1.3 Scope and Current Boundary	6
2 Methodology and Design	7
2.1 Development Methodology	7
2.2 System Architecture	7
2.2.1 High-Level Design	7
2.2.2 Implemented Foundation Flow	8
2.2.3 Planned End-to-End Flow	8
2.2.4 Technical Stack	10
2.3 Implementation Details	10
2.3.1 Manifest and Background Module	10
2.3.2 Local Embedding Module	10
2.3.3 Reproducible Asset Preparation	10
2.3.4 Shared Contracts and Modularity	11
2.3.5 Planned Capture and Extraction Module	11
2.3.6 Planned Storage Module	12
2.3.7 Planned Search and Canvas Modules	12
2.3.8 Storage Lifecycle and Portability	12
3 Results and Evaluation	13
3.1 Evaluation Methodology	13
3.2 Foundation Results	13
3.3 Analysis	15
3.3.1 Objectives Met in the Current Milestone	15
3.3.2 Engineering Challenges and Resolutions	15
3.3.3 Current Limitations	15
4 Conclusion	17
4.1 Summary of Work	17
4.2 Key Findings	17
4.3 Future Work and Recommendations	17
4.4 Final Remarks	18
Bibliography	19

Executive Summary

Meridian: Latent Cartographer is a local-first spatial knowledge engine proposed as a Chromium Manifest V3 extension. It addresses the fragmentation created when students, researchers, and developers work across many browser tabs, articles, documentation pages, PDFs, Markdown notes, and code snippets. Rather than presenting saved material as a flat bookmark list, Meridian is intended to represent each item by its semantic meaning and place related items on an interactive two-dimensional canvas.

The project is designed around on-device processing. Text is converted into semantic embeddings by the `Xenova/all-MiniLM-L6-v2` model through `Transformers.js` and `ONNX Runtime WebAssembly`. Search will compare a locally generated query vector with stored item and passage vectors. `IndexedDB` is planned for searchable records, while the `Origin Private File System (OPFS)` is planned for larger binary assets. `UMAP` and a collision-correction pass will later convert item-level vectors into readable canvas coordinates.

Development is incremental. The completed Milestone 1 Foundation Prototype establishes a loadable Manifest V3 extension, a React and TypeScript application, a dedicated inference Web Worker, reproducible local model packaging, and real 384-dimensional embedding generation without external runtime networking. Webpage capture, persistence, semantic search, and the spatial canvas remain subsequent milestones. This report therefore separates implemented results from approved design and future work.

Introduction

1.1 Background and Context

Digital research rarely occurs in a single application or document. A typical session may include dozens of browser tabs, downloaded papers, technical documentation, local notes, code fragments, and tables. Browsers make these sources easy to open, but their tab bars and bookmark managers preserve little of the user’s conceptual context. Titles are shortened, similar pages are difficult to distinguish, and a returning user must often reread several sources before remembering why each one mattered.

Keyword search only partly addresses this problem. A user may remember an idea but not the words used by the original author. For example, a query about “storing browser data without a server” should be able to retrieve a source discussing IndexedDB or offline persistence even if the query and source share few literal terms. Dense sentence embeddings provide a practical mechanism for this form of semantic retrieval. The MiniLM model selected for Meridian maps text to a 384-dimensional vector space in which semantically related passages can be compared using cosine similarity [WWD+20; Hug].

Many semantic-search products depend on remote inference. Sending research text and search queries to a cloud service creates privacy, latency, cost, and availability concerns. Transformers.js makes it possible to execute compatible models locally in the browser using ONNX Runtime and WebAssembly [Hug; Mic]. Meridian uses this capability to keep its core embedding and retrieval workflow on the user’s device.

1.1.1 Problem Statement

Students, researchers, and developers experience cognitive overload and context loss when useful information is spread across flat browser-tab lists, bookmarks, local documents, and code references. Existing organization methods depend heavily on filenames, URLs, folder placement, or exact keywords. They do not adequately preserve the semantic relationship between sources or the passage that made a source relevant.

The project must therefore provide a private and responsive method to capture heterogeneous research material, extract meaningful structure, index the material semantically, persist it locally, and support visual as well as textual retrieval. It must operate within Manifest V3 lifecycle restrictions and avoid a persistent server or paid inference API.

1.1.2 Intended Users

- **Students** conducting literature reviews, studying documentation, preparing reports, or collecting sources across several subjects.

- **Researchers** organizing papers, articles, notes, and supporting evidence by conceptual relationship rather than folder membership alone.
- **Developers** saving documentation, technical articles, debugging references, code blocks, and structured tables for later retrieval.

1.1.3 Proposed User Journey

The target workflow is as follows:

1. The user encounters a useful webpage, PDF, Markdown note, or code reference.
2. The user explicitly saves or imports the source into Meridian.
3. Meridian extracts and normalizes the source content.
4. A local model generates semantic embeddings for the item and, in Deep Focus mode, its individual chunks.
5. Searchable records are stored in IndexedDB and large assets are stored in OPFS.
6. The item appears as a card on the research canvas.
7. Related items are placed in nearby semantic neighbourhoods.
8. The user submits a natural-language query.
9. Meridian ranks stored items and passages using cosine similarity.
10. The user inspects the matching passage and reopens the original source.

At the present stage, the local embedding portion of this journey is implemented. The other steps are planned work and are not reported as completed functionality.

1.2 Project Objectives

1. **Establish local semantic inference.** Build a Manifest V3 extension that produces real, normalized, 384-dimensional embeddings through a dedicated WASM Web Worker, with remote model loading disabled.
2. **Implement reliable research capture.** Extract useful text and available structure from active webpages, and later support PDFs, Markdown notes, and explicitly entered code snippets.
3. **Provide semantic retrieval.** Embed natural-language queries locally and rank saved item and chunk vectors using cosine similarity.

4. **Create a spatial workspace.** Project item-level embeddings onto a stable two-dimensional canvas that supports pan, zoom, selection, pinning, manual placement, and source inspection.
5. **Preserve privacy and offline availability.** Avoid a project backend and paid AI APIs for capture, embedding, storage, and search. Package required model and WASM assets with the local extension deliverable.
6. **Control storage growth.** Use quantized vectors, separate searchable and binary storage, on-demand asset loading, and understandable cleanup rules while protecting pinned research.
7. **Support reproducible team development.** Pin dependencies and model assets, define shared TypeScript contracts, separate module ownership, and require independently reviewable feature branches.

1.3 Scope and Current Boundary

The completed scope is the technical foundation: extension loading, toolbar launch, typed worker communication, local model preparation, WASM inference, a diagnostic interface, and offline verification. The current interface accepts short text and displays embedding metadata and values.

Webpage capture, IndexedDB and OPFS persistence, semantic search across stored content, PDF and Markdown ingestion, UMAP projection, Konva-based canvas rendering, lifecycle cleanup, and workspace export remain within the approved project scope but are not yet implemented. OCR for scanned PDFs and full language-specific Abstract Syntax Tree analysis are outside the initial scope.

Methodology and Design

2.1 Development Methodology

The project follows an incremental, risk-first approach. The highest-risk assumption was whether a Chromium extension could load a pinned model locally, execute it through WASM in a worker, and generate meaningful embeddings without runtime network access. Milestone 1 was therefore implemented before capture, storage, and visual design were expanded.

Each milestone has an observable acceptance condition. Shared interfaces are established before parallel module work so that extraction, storage, inference, and UI code can be developed independently. Existing academic repository folders and the instructor's C++ sample are preserved. New product code is isolated under `code/extension/`.

2.2 System Architecture

2.2.1 High-Level Design

Meridian separates orchestration, extraction, computation, persistence, retrieval, layout, and presentation.

1. A Manifest V3 background service worker responds to extension events and coordinates browser actions.
2. Content scripts will extract information from explicitly selected webpages.
3. A React application will present the capture state, search controls, inspector, and spatial workspace.
4. Dedicated Web Workers perform embedding and later layout computation away from the main interface thread.
5. IndexedDB will hold metadata, text chunks, embeddings, jobs, layout, and settings.
6. OPFS will hold screenshots and other large binary assets.

The service worker is intentionally not treated as a persistent computation host. Manifest V3 service workers can be suspended during inactivity, so durable state will be stored and longer computation will run from an extension page or an appropriate offscreen context. This design follows the event-driven service-worker model required by Chromium extensions [Gooa; Goob].

2.2.2 Implemented Foundation Flow

The implemented runtime sequence is:

1. The user clicks the Meridian toolbar action.
2. The background service worker uses `chrome.tabs.create` and an internal `chrome.runtime.getURL` URL to open the application page.
3. React collects a non-empty short-text input.
4. The embedding client creates a unique request identifier and posts a typed request to a dedicated Web Worker.
5. The worker loads the packaged tokenizer, quantized ONNX model, and compatible WASM runtime from the extension origin.
6. Transformers.js executes feature extraction with mean pooling and normalization.
7. The worker verifies that the result contains exactly 384 finite values.
8. The `Float32Array` buffer is transferred back to the client.
9. The diagnostic interface displays the model, dimensions, backend, elapsed time, vector norm, and complete vector values.

Inference requests are serialized so that multiple UI requests do not compete for the same WASM session. The initialized model remains available while the application page and worker remain open.

2.2.3 Planned End-to-End Flow

The complete processing path will be:

1. capture or import a research source;
2. extract and normalize its content;
3. create a lightweight representation or Deep Focus chunks;
4. generate local embeddings;
5. quantize and persist vectors and metadata;
6. update semantic coordinates;
7. render searchable cards; and
8. retrieve ranked items and passages for a local query.

Area	Technology	Role
Platform	Chromium Manifest V3	Extension lifecycle, toolbar action, browser integration, and content-script execution.
Language	TypeScript	Shared contracts and safer communication across extension contexts.
Interface	React 19	Current diagnostic UI and planned workspace controls.
Build	Vite 8	Production bundling for the application, service worker, and inference worker.
Local ML	Transformers.js 3.8.1	Browser-side model loading, tokenization, pooling, and inference.
Model	Xenova/all-MiniLM-L6-v2	Normalized 384-dimensional semantic embeddings.
Runtime	ONNX Runtime Web and WASM	Local CPU execution of the quantized ONNX model.
Search	Cosine similarity	Planned ranking of query, item, and chunk vectors.
Searchable storage	IndexedDB	Planned items, chunks, vectors, jobs, layout, and settings.
Binary storage	OPFS	Planned WebP previews and optional snapshots.
Extraction	Mozilla Readability	Planned readable webpage extraction with a fallback path.
PDF processing	PDF.js	Planned local PDF text extraction and page references.
Spatial layout	UMAP-JS	Planned conversion from 384 dimensions to 2D coordinates.
Canvas	Konva.js	Planned interactive card rendering, pan, zoom, and selection.
Browser automation	Playwright Chromium	Isolated foundation verification in a fresh profile.

Table 2.1: Meridian technology stack and responsibilities

2.2.4 Technical Stack

2.3 Implementation Details

2.3.1 Manifest and Background Module

The manifest declares a module-based background service worker, toolbar action, minimum Chrome version 116, and a restrictive Content Security Policy. The current policy permits scripts, workers, styles, images, and connections only from the extension itself, with the WASM evaluation capability required by ONNX Runtime. No active-page capture permission has been added during the foundation milestone because capture is not yet implemented.

The background module currently performs one responsibility: opening the Meridian application page when the toolbar action is clicked. Keeping it small reduces dependence on transient service-worker memory and leaves persistent jobs for the later storage layer.

2.3.2 Local Embedding Module

The embedding module uses `Xenova/all-MiniLM-L6-v2`, a browser-compatible conversion of the MiniLM sentence-transformer model [WWD+20; Xen]. It requests the quantized ONNX weights and explicitly selects the WASM backend. One WASM thread is used for compatibility without cross-origin isolation, and ONNX proxy mode is disabled because inference already runs in a dedicated worker.

The worker configures the Transformers.js environment to permit local files and disallow remote models. Browser and filesystem model caches are disabled for the foundation check so that a warm external cache cannot conceal a missing packaged asset. A worker-level fetch guard rejects any request whose origin differs from the installed extension origin.

Mean pooling produces a single vector for the input text, and normalization gives the vector an approximately unit Euclidean norm. For normalized vectors, cosine similarity can be calculated efficiently using a dot product:

$$\text{cosine}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2}. \quad (2.1)$$

The current UI restricts input to 12,000 characters as a guard, but this does not mean the entire input always fits the model context. Proper token-aware chunking is scheduled for the Deep Focus milestone.

2.3.3 Reproducible Asset Preparation

The model is pinned to revision `751bff37182d3f1213fa05d7196b954e230abad9`. The project records SHA-256 checksums for the model configuration, tokenizer files, vocabulary, special-token mapping, and quantized ONNX model. The preparation script downloads these public assets during development, writes them atomically, and copies the matching ONNX Runtime WASM glue and binary from the

locked dependency. The production build verifies the checksums and runtime match before bundling.

Downloaded models, dependency folders, build output, caches, and generated verification results are ignored by Git. A teammate reproduces them from `package-lock.json`, `assets.lock.json`, and the preparation script rather than committing large generated files.

The model and Transformers.js use the Apache License 2.0, and ONNX Runtime uses the MIT License. Corresponding attribution files are placed beside prepared assets.

2.3.4 Shared Contracts and Modularity

The foundation defines common TypeScript contracts rather than allowing modules to invent incompatible records.

Contract	Purpose
<code>ExtractedDocument</code>	Represents a webpage, PDF, Markdown note, or snippet with title, source, normalized text, and ordered paragraph, heading, code, or table blocks.
<code>CanvasItem</code>	Represents a workspace card with source metadata, excerpt, canvas position, pin state, and processing status.
<code>EmbeddingRequest</code>	Carries request identity and input text from a client to the worker.
<code>EmbeddingResponse</code>	Represents progress, successful result, or failure.
<code>EmbeddingResult</code>	Carries the model identity, 384 dimensions, WASM backend, normalized vector, and elapsed time.

Table 2.2: Shared module contracts

Module ownership is separated into background, content, workers, extraction, storage, search, layout, UI, and shared contracts. Extraction work can therefore return `ExtractedDocument` records while UI work uses `CanvasItem` records. Changes to the manifest, package files, build configuration, embedding worker, or shared contracts require coordination because these files affect multiple modules.

2.3.5 Planned Capture and Extraction Module

Active-page capture will be explicit. Mozilla Readability will isolate the main article where possible, while a safe DOM-text fallback will handle pages for which article extraction fails [Mozb]. Structured extraction will preserve block order, headings, paragraphs, `pre/code` blocks, and tables. A repeated source will offer an update instead of silently creating a duplicate.

Lightweight mode will retain representative text, metadata, one available preview, and one item vector. Deep Focus mode will retain full text and create proposed 256-token windows with a 50-token overlap. Actual chunk boundaries will use the model tokenizer and account for special tokens and input limits.

Local PDFs will later be processed through PDF.js [Moza]. Markdown import will preserve headings, code fences, tables, and note text. Scanned PDFs without an extractable text layer will receive an explanatory message; OCR is outside the initial scope.

2.3.6 Planned Storage Module

IndexedDB will act as the hot searchable layer for item metadata, chunks, vectors, quantization parameters, model identifiers, processing jobs, coordinates, and settings [Mozc]. OPFS will hold larger screenshots and optional snapshots that are loaded on demand [Mozd].

The planned job states are `queued`, `extracting`, `embedding`, `ready`, and `failed`. Persisting these states allows recovery after a service worker or application interruption and prevents unsafe duplicate retries.

SQ8 quantization is planned for stored embeddings. The implementation must preserve the scale or range values required to reconstruct similarity scores and must compare retrieval quality with the original floating-point vectors before adopting compression. The project target is at least 70% vector-storage reduction, but this target has not yet been measured.

2.3.7 Planned Search and Canvas Modules

The search module will embed the query using the same local model and rank stored item and chunk vectors with cosine similarity. Search ranking will use high-dimensional vectors rather than distance on the 2D map. The selected result will show its relevant passage and source.

The layout module will use UMAP to project item-level vectors to two dimensions [MHM18; MC]. A collision-correction pass will reduce card overlap. Coordinates will be persisted, manually placed or pinned cards will be respected, and users will request major reorganization explicitly. Konva.js is planned for the interactive canvas [Kon]. Viewport culling and controlled image loading will reduce rendering and memory pressure.

2.3.8 Storage Lifecycle and Portability

Pinned items will be protected from automatic cleanup. Unpinned preview assets are proposed to become eligible for cleanup after seven inactive days while text and searchable vectors remain. Thirty-day centroid-only reduction is optional and disabled initially because it would reduce passage-level search quality.

The planned portability layer includes versioned JSON export and import with preservation of content and coordinates. A standalone HTML export will initially act as an inert viewer. Including local AI search inside that exported file requires a separate model-distribution decision, and captured scripts must never execute in the viewer.

Results and Evaluation

3.1 Evaluation Methodology

Milestone 1 was evaluated as an actual extension rather than only as isolated JavaScript. TypeScript compilation and the Vite production build first confirmed that the application, background worker, and inference worker bundled successfully. The build also verified model checksums and compatibility of the copied WASM runtime.

An integration scenario then loaded the built extension into a fresh Playwright Chromium profile. Before the application page was opened, browser offline mode was enabled. External browser traffic was additionally directed to an unreachable local proxy, DNS resolution was disabled, and background networking was suppressed. This provided multiple independent controls against accidental external access.

Three short sentences were embedded:

1. “A cat is sleeping on a warm windowsill.”
2. “A kitten is resting beside the sunny window.”
3. “Database transactions provide atomicity and isolation.”

The evaluation checked that each result contained 384 finite values, was approximately unit normalized, contained non-constant values, and differed across unrelated inputs. It also confirmed that related sentences received greater cosine similarity than an unrelated pair. Every model and runtime request observed during inference used a `chrome-extension://` URL. A separate external navigation probe failed under the same browser restrictions.

No student usability study has yet been conducted. The proposed pilot involving engineering students remains future work.

3.2 Foundation Results

The first-call time includes model and WASM initialization. Later calls reuse the model held by the worker and should not be interpreted as complete search latency. The proposed 20 ms retrieval target includes a different workload and will be measured after query embedding, stored-vector scoring, result selection, and UI response have been integrated.

Measure	Project target or rule	Observed foundation result
Embedding dimensions	Exactly 384	384 finite values
Embedding norm	Approximately 1 after normalization	1.000000 in the diagnostic result
Runtime backend	Local WASM	WASM
First short embedding	No final requirement established	Approximately 360.2 ms
Later short embeddings	No final requirement established	Approximately 7.7 ms and 7.6 ms
Related-sentence similarity	Greater than unrelated comparison	Approximately 0.589
Unrelated-sentence similarity	Lower than related comparison	Approximately -0.069
External inference requests	None	None observed
External navigation probe	Must fail in offline scenario	Blocked
Semantic search latency	Median ≤ 20 ms over 200+ artifacts	Not yet measurable
Canvas smoothness	55–60 FPS with 100+ active nodes	Not yet measurable
Tab ingestion	≤ 1.5 s per tab	Not yet measurable
Vector storage reduction	At least 70% compared with Float32	Not yet measurable

Table 3.1: Foundation observations and outstanding project targets

3.3 Analysis

3.3.1 Objectives Met in the Current Milestone

The foundation objective was met. A real quantized MiniLM model generated normalized 384-dimensional vectors inside a dedicated extension worker. The extension opened through a Manifest V3 background action, and the model and WASM assets were served from the installed extension during the offline scenario.

Semantic comparison behaved in the expected direction: the cat and kitten sentences were substantially closer than the cat and database sentences. This does not constitute a complete retrieval-quality study, but it rules out a constant or obviously unrelated mock result and supports continuation with the chosen embedding pipeline.

3.3.2 Engineering Challenges and Resolutions

- **Offline asset loading.** Transformers.js normally supports hosted model and runtime paths. Meridian instead pins and packages the model and compatible WASM files, disables remote models, verifies checksums, and restricts worker fetches.
- **Manifest V3 lifecycle.** Persistent model state is not placed in the background service worker. The current application page owns the inference worker, while later job state will be persisted so that browser suspension does not lose progress.
- **Reliable verification.** A fresh profile, disabled model caches, browser offline mode, an unreachable proxy, disabled DNS, observed request URLs, and an external negative control reduce the possibility that cached or remote files caused an apparent pass.
- **Dependency advisory.** The Node-side `sharp` dependency inherited by Transformers.js was overridden to version 0.35.0. The dependency audit subsequently reported no known vulnerabilities. The extension's runtime path uses the browser WASM bundle rather than Sharp or ONNX Runtime Node.

3.3.3 Current Limitations

- The current UI is an embedding diagnostic, not the complete Meridian workspace.
- Webpage capture, structured extraction, duplicate handling, and screenshots are not implemented.
- IndexedDB and OPFS schemas are not implemented, so research items do not yet persist.

- Semantic retrieval across saved content and passage inspection are not implemented.
- The spatial canvas, UMAP projection, collision correction, and viewport culling are not implemented.
- PDF, Markdown, and snippet ingestion are not implemented.
- The current input guard is character-based; token-aware document chunking remains future work.
- The extension is locally deployed as an unpacked build and is not published in the Chrome Web Store.
- Reported times are observations from one environment, not product-wide guarantees.

Conclusion

4.1 Summary of Work

The Meridian project has progressed from an architectural proposal to a working foundation prototype. The current implementation establishes a Chromium Manifest V3 application using React, TypeScript, and Vite; a background action that opens the application; a typed client and dedicated inference worker; pinned and checksum-verified MiniLM assets; local ONNX WASM execution; and a reproducible offline verification workflow.

The implementation is contained within `code/extension/`, preserving the instructor's repository structure, academic reports, journals, and existing C++ example. The foundation is recorded in commit `1253aae` on the remote `codex/foundation` branch. At the latest recorded repository review, this branch had not yet been merged into the default `master` branch.

4.2 Key Findings

- **Local browser inference is feasible.** A packaged quantized MiniLM model can generate real 384-dimensional semantic embeddings inside a Manifest V3 extension using WASM and without runtime cloud inference.
- **Worker separation supports the intended architecture.** Model computation can remain outside the React UI and transient background service worker while still exposing a small typed client interface to future search and ingestion modules.
- **Offline claims require explicit controls.** Packaging files alone is not enough. Remote-model configuration, extension CSP, origin-restricted fetches, fresh-profile execution, cache control, and negative network checks together provide stronger evidence.
- **Measured foundation timings do not prove final search targets.** Initialization, query embedding, scoring, UI response, ingestion, and layout are different measurements and must be reported separately in later milestones.

4.3 Future Work and Recommendations

1. **Merge and stabilize the foundation.** Review the `codex/foundation` branch and merge it into `master` before parallel feature branches begin.
2. **Implement capture and persistence.** Add explicit active-page capture, Readability extraction, structured blocks, duplicate handling, an IndexedDB

schema, initial OPFS assets, persistent job states, and browser-restart recovery.

3. **Deliver the first complete workflow.** Connect stored items to the embedding worker, cosine-similarity search, a basic canvas, an inspector, result highlighting, and source reopening.
4. **Add Deep Focus ingestion.** Implement tokenizer-aware overlapping chunks, PDF and Markdown import, manual snippets, source references, and passage-level retrieval.
5. **Build the semantic map.** Add UMAP projection, collision correction, stable coordinates, pinned and manually positioned cards, explicit reorganization, and viewport culling.
6. **Complete storage and portability.** Add understandable cleanup controls, pinned-item protection, versioned JSON export and import, and a safe standalone HTML viewer.
7. **Evaluate at project scale.** Measure retrieval quality and latency on at least 200 artifacts, canvas responsiveness with at least 100 nodes, actual storage reduction, cold and warm inference, ingestion time, and usability with an agreed pilot protocol.
8. **Prepare distribution.** Produce a reproducible extension package and complete project documentation before considering Chrome Web Store publication.

4.4 Final Remarks

Milestone 1 does not yet solve the entire research-organization problem, but it validates the most important technical dependency beneath the proposed solution: meaningful local semantic inference inside the browser. The next phase can now build capture and storage against a real embedding service rather than a mock. Maintaining the distinction between implemented, verified, approved, and planned capabilities will keep the project report accurate as Meridian progresses toward its spatial research workspace.

Bibliography

- [Gooa] Google Chrome Developers. *Extension Service Workers*. URL: <https://developer.chrome.com/docs/extensions/develop/concepts/service-workers> (visited on 09/16/2026) (cit. on p. 7).
- [Goob] Google Chrome Developers. *Offscreen Documents in Chrome Extensions*. URL: <https://developer.chrome.com/docs/extensions/reference/api/offscreen> (visited on 09/16/2026) (cit. on p. 7).
- [Hug] Hugging Face. *Transformers.js Documentation*. URL: <https://huggingface.co/docs/transformers.js/> (visited on 09/16/2026) (cit. on p. 4).
- [Kon] Konva Contributors. *Konva.js Documentation*. URL: <https://konvajs.org/docs/> (visited on 09/16/2026) (cit. on p. 12).
- [MC] Leland McInnes and Contributors. *UMAP-JS*. URL: <https://github.com/PAIR-code/umap-js> (visited on 09/16/2026) (cit. on p. 12).
- [MHM18] Leland McInnes, John Healy, and James Melville. “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction”. In: *arXiv preprint arXiv:1802.03426* (2018). arXiv: 1802.03426 (cit. on p. 12).
- [Mic] Microsoft. *ONNX Runtime Web*. URL: <https://onnxruntime.ai/docs/get-started/with-javascript/web.html> (visited on 09/16/2026) (cit. on p. 4).
- [Moza] Mozilla. *PDF.js*. URL: <https://mozilla.github.io/pdf.js/> (visited on 09/16/2026) (cit. on p. 12).
- [Mozb] Mozilla. *Readability*. URL: <https://github.com/mozilla/readability> (visited on 09/16/2026) (cit. on p. 11).
- [Mozc] Mozilla Developer Network. *IndexedDB API*. URL: https://developer.mozilla.org/en-US/docs/Web/API/IndexedDB_API (visited on 09/16/2026) (cit. on p. 12).
- [Mozd] Mozilla Developer Network. *Origin Private File System*. URL: https://developer.mozilla.org/en-US/docs/Web/API/File_System_API/Origin_private_file_system (visited on 09/16/2026) (cit. on p. 12).
- [WWD+20] Wenhui Wang, Furu Wei, Li Dong, et al. “MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers”. In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 5776–5788 (cit. on pp. 4, 10).
- [Xen] Xenova. *all-MiniLM-L6-v2 for Transformers.js*. URL: <https://huggingface.co/Xenova/all-MiniLM-L6-v2> (visited on 09/16/2026) (cit. on p. 10).